

EXERCICE 1 — Forward Pass et analyse des paramètres

Question 1.1

Architecture du réseau : $\text{Input}(3) \rightarrow \text{Dense}(2, \text{ReLU}) \rightarrow \text{Dense}(1, \text{Sigmoid})$

Données :

- Entrées : $\mathbf{x}=[0,5; 1; 2]$
 - Couche 1 : $\mathbf{W}_{[1]}=[0,20,30,40,50,10,2]$, $\mathbf{b}_{[1]}=[0,1; 0,2]$
 - Couche 2 : $\mathbf{W}_{[2]}=[0,70,3]$, $\mathbf{b}_{[2]}=[0,1]$
-

Étape 1 : Calcul de $\mathbf{z}_{[1]}=\mathbf{x} \cdot \mathbf{W}_{[1]}+\mathbf{b}_{[1]}$

Premier neurone caché : $z_{1[1]}=(0,5 \times 0,2)+(1 \times 0,3)+(2 \times 0,4)+0,1$ $z_{1[1]}$
 $=0,1+0,3+0,8+0,1=1,3$

Deuxième neurone caché : $z_{2[1]}=(0,5 \times 0,5)+(1 \times 0,1)+(2 \times 0,2)+0,2$ $z_{2[1]}$
 $=0,25+0,1+0,4+0,2=0,95$

$\mathbf{z}_{[1]}=[1,3; 0,95]$

Étape 2 : Application de la fonction d'activation ReLU

$\text{ReLU}(z)=\max(0,z)$

$a_{1[1]}=\text{ReLU}(1,3)=1,3$ $a_{2[1]}=\text{ReLU}(0,95)=0,95$

$\mathbf{a}_{[1]}=[1,3; 0,95]$

Étape 3 : Calcul de $\mathbf{z}_{[2]}=\mathbf{a}_{[1]} \cdot \mathbf{W}_{[2]}+\mathbf{b}_{[2]}$

$z_{[2]}=(1,3 \times 0,7)+(0,95 \times 0,3)+0,1$ $z_{[2]}=0,91+0,285+0,1=1,295$

$$z[2]=1,295$$

Étape 4 : Application de la fonction Sigmoid

$$\sigma(z)=\frac{1}{1+e^{-z}}$$

$$y=\frac{1}{1+e^{-1,295}}$$

Utilisation de l'approximation $e^{-1} \approx 0,368$:

$$e^{-1,295} = e^{-1} \times e^{-0,295} \approx 0,368 \times 0,7445 \approx 0,274$$

$$y = \frac{1}{1+0,274} = \frac{1}{1,274} \approx \mathbf{0,785}$$

$$y \approx 0,785$$

Question 1.2

Formule : Paramètres = (entrées × neurones) + neurones

Couche 1 (Dense, 2 neurones)

- Entrées : 3, Neurones : 2
- Poids : $3 \times 2 = 6$
- Biais : 2
- **Total : $6 + 2 = 8$ paramètres**

Couche 2 (Dense, 1 neurone)

- Entrées : 2, Neurones : 1
- Poids : $2 \times 1 = 2$
- Biais : 1
- **Total : $2 + 1 = 3$ paramètres**

Nombre total de paramètres entraînables = $8 + 3 = \mathbf{11}$

Question 1.3

Classe prédite

La sortie $y \approx 0,785$ représente la probabilité d'appartenance à la classe positive. Avec un seuil de décision de 0,5 :

$$0,785 > 0,5$$

Le réseau prédit la classe **1** (classe positive)

Justification : La probabilité estimée dépasse le seuil de décision, ce qui conduit à l'attribution de la classe positive selon la règle de décision standard en classification binaire.

Mécanisme d'apprentissage pour ajuster les poids

Le mécanisme est la **rétropropagation du gradient** (*Backpropagation*).

Principe : On calcule d'abord l'erreur entre la sortie prédite et la sortie attendue (la classe opposée) à l'aide d'une fonction de perte (par exemple l'entropie croisée binaire). Cette erreur est ensuite propagée en arrière à travers le réseau, couche par couche, en appliquant la règle de dérivation en chaîne pour calculer les gradients de la perte par rapport à chaque paramètre (poids et biais). L'optimiseur (Adam, SGD, etc.) met ensuite à jour les poids dans la direction opposée au gradient, avec un pas contrôlé par le taux d'apprentissage. Ce processus itératif minimise progressivement la fonction de perte jusqu'à convergence.

EXERCICE 2 — Construction d'un modèle Keras

Question 2.1 (3 points)

Python

Copy

```

import numpy as np
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras

# =====
# 1. PRÉPARATION DES DONNÉES
# =====

# Tableaux NumPy X_train et y_train
X_train = np.array([
    [8, 1200, 1, 4.5],
    [10, 1200, 2, 4.5],
    [14, 1250, 3, 4.5],
    [18, 1250, 4, 4.5],
    [24, 1300, 5, 4.5],
    [32, 1300, 6, 5.2],
    [38, 1350, 7, 5.2],
    [36, 1350, 8, 5.2],
    [28, 1300, 9, 4.5],
    [20, 1300, 10, 4.5],
    [12, 1250, 11, 4.5],
    [7, 1250, 12, 4.5]
], dtype=np.float32)

y_train = np.array([
    580, 520, 430, 350, 310,
    480, 620, 590, 380, 330,
    490, 600
], dtype=np.float32)

# Normalisation avec StandardScaler
scaler_X = StandardScaler()
X_train_scaled = scaler_X.fit_transform(X_train)

scaler_y = StandardScaler()
y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1, 1)).flatten()

# =====
# 2. CONSTRUCTION DU MODÈLE SÉQUENTIEL
# =====

model = keras.Sequential([
    keras.layers.Dense(16, activation='relu', input_shape=(4,)),
    keras.layers.Dense(8, activation='relu'),
    keras.layers.Dense(1, activation='linear')
])

# =====
# 3. COMPILATION
# =====

model.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=['mae']
)

# =====
# 4. ENTRAÎNEMENT

```

```
# =====

model.fit(X_train_scaled, y_train_scaled, epochs=100, verbose=0)

# =====
# 5. PRÉDICTION POUR LES SCÉNARIOS A ET B
# =====

# Scénario A
scenario_A = np.array([[35, 1400, 7, 5.2]], dtype=np.float32)
scenario_A_scaled = scaler_X.transform(scenario_A)
pred_A_scaled = model.predict(scenario_A_scaled, verbose=0)
pred_A = scaler_y.inverse_transform(pred_A_scaled)

# Scénario B
scenario_B = np.array([[9, 1300, 12, 4.5]], dtype=np.float32)
scenario_B_scaled = scaler_X.transform(scenario_B)
pred_B_scaled = model.predict(scenario_B_scaled, verbose=0)
pred_B = scaler_y.inverse_transform(pred_B_scaled)

print(f"Scénario A - Consommation prédite : {pred_A[0][0]:.2f} MWh")
print(f"Scénario B - Consommation prédite : {pred_B[0][0]:.2f} MWh")
```

Question 2.2

Pourquoi la normalisation est indispensable

Table

Variable	Plage de valeurs	Échelle relative
Température	[7; 38]	Faible
Foyers	[1200; 1400]	Très élevée
Mois	[1; 12]	Faible
Tarif	[4,5; 5,2]	Très faible

Analyse : La variable *Foyers* possède une échelle environ **100 fois supérieure** aux autres variables. Sans normalisation, les poids associés à cette variable domineront totalement le calcul des sommes pondérées lors du forward pass. L'optimiseur Adam convergera de manière déséquilibrée : les gradients pour les poids des petites échelles seront négligeables

comparés à ceux de la variable *Foyers*. Le réseau apprendra principalement à partir de cette seule variable, ignorant l'impact crucial de la température et du tarif sur la consommation électrique.

StandardScaler centre les données (moyenne ≈ 0) et les réduit (écart-type ≈ 1), garantissant que toutes les variables contribuent équitablement à l'apprentissage.

Nombre total de paramètres entraînables

Architecture : $4 \rightarrow \text{Dense}(16) \rightarrow \text{Dense}(8) \rightarrow \text{Dense}(1)$

Table

Couche	Entrées	Neurones	Poids	Biais	Total
Dense(16)	4	16	$4 \times 16 = 64$	16	80
Dense(8)	16	8	$16 \times 8 = 128$	8	136
Dense(1)	8	1	$8 \times 1 = 8$	1	9

Total = $80 + 136 + 9 = 225$ paramètres entraînables

Question 2.3

Cohérence des prédictions

Scénario A (35°C, 1400 foyers, mois 7, 5,2 DA/kWh) :

La consommation prédite devrait être **élevée** (environ 600 -650 MWh). Ce scénario correspond à un mois d'été très chaud (35°C), où la demande en climatisation est maximale. Le nombre de foyers (1400) dépasse le maximum historique (1350), ce qui amplifie la consommation. Le tarif élevé (5,2 DA/kWh) correspond aux périodes de forte

demande (juin-août). Cette prédiction est cohérente avec les données de juillet (38°C , 1350 foyers → 620 MWh).

Scénario B (9°C, 1300 foyers, mois 12, 4,5 DA/kWh) :

La consommation prédite devrait être **modérée** (environ 500 -550 MWh). Bien qu'il s'agisse d'un mois d'hiver (9°C), la température reste relativement douce. Le nombre de foyers standard (1300) et le tarif bas (4,5 DA/kWh) sont cohérents avec les données de janvier/février. Le chauffage électrique est utilisé mais la demande reste inférieure aux pics estivaux.

Avis argumenté :

Ce modèle **n'est pas suffisant** pour un déploiement en production chez Sonelgaz.

Premièrement, le volume de données est critique : seulement **12 observations** pour **225 paramètres** et 4 variables explicatives est extrêmement insuffisant. Le rapport observations/paramètres ($12/225 \approx 0,05$) est bien en dessous du seuil minimal requis, ce qui entraîne un risque majeur d'overfitting et une incapacité à généraliser. **Deuxièmement**, des variables essentielles sont absentes : le taux d'occupation des foyers, le type de chauffage dominant (électrique vs gaz), les jours fériés, et les données météorologiques horaires. **Troisièmement**, l'architecture MLP simple ne capture pas les dépendances temporelles saisonnières. **Améliorations concrètes** : (1) Collecter au minimum 3 à 5 ans de données mensuelles (36-60 observations) avec des variables explicatives complémentaires ; (2) Remplacer le MLP par un modèle **LSTM** ou ajouter un encodage cyclique du mois pour modéliser explicitement la saisonnalité.

EXERCICE 3 — Diagnostic d'entraînement et régularisation ()

Question 3.1

Diagnostic des modèles

Table

Modèle	Train Loss	Val Loss	Écart	Diagnostic	Justification
X	0,42	0,45	0,03	Underfitting	Les deux pertes sont élevées et très proches. Le modèle est trop simple (biais élevé) pour capturer la complexité des données.
Y	0,02	0,85	0,83	Overfitting	Écart massif entre train et validation. Le modèle mémorise le bruit des données d'entraînement mais échoue à généraliser.
Z	0,05	0,09	0,04	Modèle satisfaisant	Les deux pertes sont faibles et proches. Bon équilibre biais-variance.

Solutions pour le modèle X (Underfitting)

Solution 1 : Augmenter la complexité du modèle Ajouter des couches cachées supplémentaires ou augmenter le nombre de neurones par couche. Le modèle actuel est trop simple pour représenter la fonction cible sous-jacente. Une architecture plus profonde permet d'apprendre des représentations hiérarchiques et non-linéaires plus riches, réduisant ainsi le biais.

Solution 2 : Augmenter le nombre d'époques Le modèle n'a probablement pas convergé. Augmenter le nombre d'itérations d'entraînement permet à l'optimiseur de trouver un minimum plus profond de la fonction de perte, améliorant les performances sur les données d'entraînement et de validation.

Solutions pour le modèle Y (Overfitting)

Solution 1 : Ajout de Dropout Intégrer des couches `Dropout(0.4)` après chaque couche cachée. Ce mécanisme désactive aléatoirement 40% des neurones à chaque itération d'entraînement, empêchant le réseau de dépendre excessivement de certains neurones spécifiques. Cela force l'apprentissage de représentations redondantes et robustes, réduisant la variance du modèle.

Solution 2 : EarlyStopping avec validation split Implémenter un callback `EarlyStopping` qui surveille la `val_loss` et arrête l'entraînement après 10 époques sans amélioration. Cela évite le sur-apprentissage sur les dernières époques où le modèle commence à mémoriser le bruit plutôt qu'à apprendre la structure générale des données.

Question 3.2

Python

Copy

```
import tensorflow as tf
from tensorflow import keras

# =====
# CONSTRUCTION DU MODÈLE AVEC RÉGULARISATION
# =====

model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(15,)),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(1, activation='sigmoid')
])

# =====
# COMPILATION
# =====

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# =====
# CALLBACK EARLY STOPPING
# =====
```

```

early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

# =====
# ENTRAÎNEMENT AVEC VALIDATION SPLIT
# =====

model.fit(
    X_train, y_train,
    epochs=200,
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop]
)

```

Question 3.3

Dropout désactivé en prédiction

Le **Dropout** est un mécanisme de régularisation stochastique qui s'applique exclusivement pendant la phase d'entraînement. Lors de la prédiction (`model.predict`), il est automatiquement désactivé car l'objectif est d'utiliser l'intégralité de la capacité du réseau pour produire une prédiction stable et déterministe. Si le Dropout restait actif en inférence, certains neurones seraient aléatoirement désactivés à chaque appel de prédiction, produisant des **résultats instables et non reproductibles** pour une même entrée. Le modèle deviendrait ainsi inutilisable en production, car la même observation recevrait des prédictions différentes à chaque exécution.

Rôle de `patience=10` et `restore_best_weights=True`

`patience=10` : Ce paramètre définit le nombre d'époques consécutives pendant lesquelles la métrique surveillée (`val_loss`) peut ne pas s'améliorer avant que l'entraînement ne soit arrêté. Une patience de 10 permet d'ignorer les fluctuations locales dues au bruit stochastique (mini-batches), tout en limitant le sur-apprentissage sur le long terme.

`restore_best_weights=True` : À la fin de l'entraînement, le modèle restaure automatiquement les poids de l'époque ayant obtenu la meilleure valeur de la métrique

surveillée (ici, la `val_loss` la plus faible), plutôt que ceux de la dernière époque. Cela garantit que le modèle final correspond au point optimal trouvé pendant l'entraînement.

Si `restore_best_weights=False` : Le modèle conserverait les poids de la **dernière époque** d'entraînement. Or, après les 10 époques sans amélioration (définies par `patience`), ces poids sont probablement sur-entraînés et correspondent à une `val_loss` supérieure au meilleur point atteint précédemment. Le modèle final serait donc moins performant en généralisation, invalidant l'intérêt même de l'EarlyStopping.